The main purpose of this research was to show how can we incorporate decision tree classifier with DMN diagram. We chose Python programming language and Camunda modeler to demonstrate our findings.

Python is an interpreted, object-oriented, high-level programming language with dynamic semantics. Its high-level build in data structures, combined with dynamic typing and dynamic binding, make it very attractive for Rapid Application Development, as well as for use as a scripting or glue language to connect existing components together [1].

Camunda modeler is a desktop application for modeling BPMN and DMN diagrams [2]. BPMN and DMN files created and read by Camunda modeler are stored in XML(Extensible Markup Language) format.

Main Python modules used in our project are scikit-learn and ElementTree XML.
Scikit-learn is open source machine learning library. For the purposes of our project we used DecisionTreeClassifier class which is capable of performing multi-class classification on a dataset[3].
ElementTree XML is module that implements a simple and efficient API for parsing and creating XML data [4].
Other honorable mentions are Numpy, Pandas (used for reading csv files), deafultdict class from module collections, random and string (used for creating simple id generator), Graphviz (used for visualization of decision tree classifier) and Progress (from which we used class Bar to create simple progress bar)

What our script does in nutshell is that it accepts dataset (for demonstration we chose Iris dataset[5]) and DMN file for input and combine them. At the moment it only supports numerical datasets and DMN diagrams with one decision table. Dataset gets prepared and runs through machine learning algorithm (decision tree classifier) from which we get decision tree and percentage of correctly classified samples (there is option to adjust tree depth to prevent overfitting, have a model that can not adjust well to new unseen data).

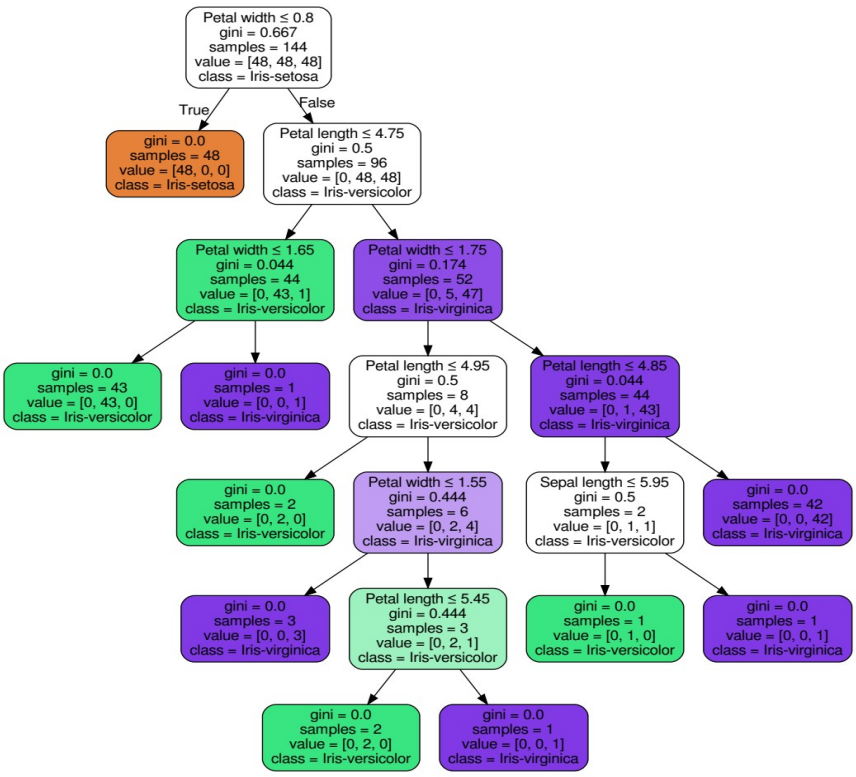


Figure 1: An example of decision tree visualized with "Graphviz" module

As we can observe from Figure 1 every sample in a leaf (node without children) took the same specific path from the root (main node) . For example, lets take node 4 (preorder tree traversal, root node is 0) which has 43 samples of class Iris-versicolor. We can conclude that every single sample in that node has “Petal width > 0.8 , Petal length <= 4.75 and Petal width <= 1.65” . This knowledge can be extracted from the decision tree and it can be used for creating DMN decision table.

DMN file (which is actually XML file) got parsed (with XML ElementTree module) and cleaned (from unwanted/old data) so we can start with inserting table columns in it which are data features as input (features used to predict certain class) and class (class that we are predicting) as output. When columns are in place we start inserting rows (rules which tell us what exact features sample has to have to be classify as one of the outputs) that are characteristics of samples in leaves. After last row has been inserted we get decision table shown in Figure 2.

First Decision

Decision_13nychf

View DRD

U	Input +			Output +	
	Petal length	Sepal length	Petal width	Class	
	double	double	double	string	Annotation
1	-	-	<= 0.8000	Iris-setosa	-
2	<= 4.7500	-	]0.8000..1.6500]	Iris-versicolor	-
3	<= 4.7500	-	> 1.6500	Iris-virginica	-
4	]4.7500..4.9500]	-	]0.8000..1.7500]	Iris-versicolor	-
5	> 4.9500	-	]0.8000..1.5500]	Iris-virginica	-
6	]4.9500..5.4500]	-	]1.5500..1.7500]	Iris-versicolor	-
7	> 5.4500	-	]1.5500..1.7500]	Iris-virginica	-
8	]4.7500..4.8500]	<= 5.9500	> 1.7500	Iris-versicolor	-
9	]4.7500..4.8500]	> 5.9500	> 1.7500	Iris-virginica	-
10	> 4.8500	-	> 1.7500	Iris-virginica	-
+	-	-	-	-	-

Figure 2: An example of decision table created from extracted decision path of every leaf from Figure 1

Every rule (row) in Figure 2 represents one leaf of the decision tree. By observing Figure 2 we can notice that some cells are left empty which means that feature is irrelevant for classification. Furthermore we can notice ranges like “]0.8000..1.6500]” which means that feature “Petal width” needs to be in range 0.8 to 1.65, excluding 0.8 and including 1.65 .

Literature:
[1] https://www.python.org/doc/essays/blurb/
[2] https://docs.camunda.org/manual/7.4/modeler/camunda-modeler/
[3] https://scikit-learn.org/stable/modules/tree.html#classification
[4] https://docs.python.org/3/library/xml.etree.elementtree.html
[5] https://archive.ics.uci.edu/ml/datasets/Iris